Opponent Modeling in Zero-Sum Games

Marcel Petráň

November 2025

1 Introduction

In multi-agent environments, the ability to reason about the behavior of others is often the defining factor between success and failure. While classical Reinforcement Learning (RL) treats the environment as a static entity, Multi-Agent Reinforcement Learning (MARL) introduces a fundamental challenge: the environment is non-stationary. As other agents learn and adapt, the transition dynamics effectively changes, rendering past experiences obsolete. To navigate this dynamic landscape, an agent must look beyond the physical state of the world and infer the hidden mental states intentions, goals, and policies of its adversaries. This process is known as Opponent Modeling (OM).

The theoretical foundations of this field trace back to the seminal work of Von Neumann and Morgenstern [17], who formalized Game Theory to analyze strategic interactions in zero-sum settings. Originally confined to strict competition, these concepts have since evolved to encompass a wide range of scenarios, including general-sum games where cooperation and coordination are possible [2]. Today, opponent modeling is a cornerstone of modern AI, finding applications in diverse domains ranging from professional sports analytics [20] and military simulations [15] to real-time strategy games [3, 11].

Despite these advances, there is a significant gap in how agents represent their opponents. Many existing approaches focus on predicting low-level actions (e.g., "Will the opponent move North or South?") or classifying opponents into broad strategic types (e.g., "Is the opponent aggressive?"). However, in complex tasks, human reasoning often operates at a higher level of abstraction. We do not merely predict an opponent's next footstep; we infer their destination.

This thesis focuses on this higher level of abstraction: Subgoal Inference. We argue that in many structured environments, an agent's policy is best understood as a sequence of intermediate objectives, or subgoals. By inferring these subgoals, a learning agent can predict the opponent's long-term trajectory, allowing for more robust counter-strategies than simple action prediction would permit.

2 Background

2.1 Explicit Sequential Subgoal Games (ESS)

In the domain of Opponent Modeling and Multi-Agent Reinforcement Learning (MARL), defining the scope of applicability is crucial. Many complex tasks can be viewed hierarchically: to achieve a final victory, an agent must first complete a series of intermediate objectives, or subgoals. A subgoal is conceptually a specific state or configuration that acts as a stepping stone toward the ultimate objective.

However, standard game taxonomies do not sufficiently capture the distinction between games with concrete, spatial subgoals and those with abstract, emergent strategic conditions. To address this ambiguity and formally delimit the class of environments where subgoal inference is tractable, we define the class of Explicit Sequential Subgoal (ESS) domains.

2.1.1 State Space Decomposition

We propose a decomposition of the state space to distinguish between the abstract global state and atomic objectives. Let the global state space $\mathcal{S}$ be a Cartesian product of two subspaces:

$$\mathcal{S} = \mathcal{P} \times \mathcal{Q} \tag{1}$$

where:

- $\mathcal{P}$ represents the subgoal feature space. These are the state variables directly related to the atomic objective completion. For example, agent's coordinates.
- $\mathcal{Q}$ represents the context feature space. These are variables that constrain the agent but do not define the immediate objective. That can be wall locations, obstacles, free space, etc.

We define the projection function $\rho : \mathcal{S} \to \mathcal{P}$ as:

$$\rho(s) = p, \quad \text{where } s = (p, q),\ p \in \mathcal{P},\ q \in \mathcal{Q} \tag{2}$$

2.1.2 Rewarding Regions

In general RL tasks, the reward function $\mathcal{R}$ often depends on the complex interaction of all features in $\mathcal{S}$ and is usually non-zero only at terminal states. In contrast, ESS games are characterized by rewards that are grounded specifically in the subgoal feature space $\mathcal{P}$.

Let $\mathcal{S}^+ \subseteq \mathcal{S}$ be the inverse image of the positive reward values:

$$\mathcal{S}^+ = \mathcal{R}^{-1}\big[(0, \infty)\big] = \{s \in \mathcal{S} \mid \mathcal{R}(s) > 0\} \tag{3}$$

2.1.3 Explicit Subgoals

We define the set of Explicit Subgoals $\mathcal{G} \subseteq \mathcal{P}$ as the image of the rewarding states under the projection ρ:

$$\mathcal{G} = \rho[\mathcal{S}^+] = \{\rho(s) \mid s \in \mathcal{S}^+\} \tag{4}$$

This formulation implies that a subgoal is not a latent variable but a verifiable configuration within the observable feature space $\mathcal{P}$.

2.1.4 Definition of an ESS Game

A game is classified as an ESS game if and only if:

- **Cardinality:** The set of subgoals $\mathcal{G}$ is finite and discrete ($|\mathcal{G}| < \infty$).

The Sequential Nature: The term "Sequential" in ESS captures the temporal structure of the agent's trajectory. While the environment may present multiple active objectives simultaneously, the agent's behavior manifests as an ordered sequence of discrete achievements ($g_1 \to g_2 \to \dots$). At any given timestep, the agent's policy is driven by the intent to reach the next immediate subgoal. Upon its completion, the agent transitions to targeting a new objective. This structure effectively decomposes the long-horizon task into a chain of shorter segments of variable duration, motivating the use of sequence modeling architectures to predict the opponent's next intent based on their history.

Example (Foraging 2.2.2):

- $\mathcal{S}$: The full grid state.
- $\mathcal{P}$: The (x, y) coordinates of the agent.
- $\mathcal{Q}$: Wall, food and opponent locations.

- $\mathcal{S}^+$: All grid configurations where the agent coincides with a food item.
- $\rho[\mathcal{S}^+]$: The specific set of (x, y) coordinates where food currently exists.

Counter-Example (Chess):

- **High-Dimensional Goal:** In Foraging, the subgoal space $\mathcal{P}$ is significantly smaller than the full state space $\mathcal{S}$ ($\dim(\mathcal{P}) \ll \dim(\mathcal{S})$). In Chess, determining a checkmate requires knowledge of the positions of the King, attacking pieces, and potential blockers.
- **Failure of Decomposition:** If we define the projection ρ to capture all necessary features for checkmate, then $\mathcal{P} \approx \mathcal{S}$. This fails the core premise of ESS, which relies on decomposing the task into a low-dimensional subgoal space and a context space.

2.2 Markov Games and the Foraging Environment

To evaluate our method, we focus on multi-agent interactions within fully observable environments. We first provide the general mathematical framework of Markov Games, and subsequently define the specific Foraging Environment used in our experiments.

2.2.1 Markov Game Formalism

We adopt the **Markov Game** (or Stochastic Game) formalism [14, 8]. This framework generalizes the Markov Decision Process (MDP) to multi-agent settings. Unlike partially observable frameworks (POMDPs), a Markov Game assumes the global state is fully accessible to all agents.

A Markov Game is defined by the tuple $\mathcal{M} = \langle \mathcal{N}, \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} \rangle$, where:

- $\mathcal{N}$ is the set of N agents.
- $\mathcal{S}$ is the global state space observed by all agents.
- $\mathcal{A} = \mathcal{A}_1 \times \cdots \times \mathcal{A}_N$ is the joint action space.
- $\mathcal{P} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ is the state transition function conditioned on joint actions.
- $\mathcal{R}_i : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function for agent i.

In this setting, the environment is non-stationary from the perspective of a single learning agent, as the transition dynamics and expected rewards depend on the unknown and potentially changing policies of other agents, π_{-i}.

2.2.2 Foraging Environment

We instantiate this framework as a two-player Foraging Game [5]. This domain fits the definition of an ESS game, where agents compete or coordinate to collect resources on a grid.

The specific tuple elements are defined as follows:

- $\mathcal{N} = \{1, 2\}$, where Agent 1 is the learning agent and Agent 2 is the opponent.
- $\mathcal{S}$ represents the global grid configuration. A state $s \in \mathcal{S}$ is a tensor of shape $H \times W \times F$, fully describing the positions of all agents and the K food items.
- $\mathcal{A}_i = \{\text{North, South, East, West}\}$ is the discrete set of movement primitives.
- $\mathcal{P}$ is deterministic:
  - Movement actions update the agent's coordinates to the adjacent cell (bounded by walls).
  - Agents automatically collect items upon entering a cell containing food.
  - Collisions prevent agents from occupying the same cell simultaneously.
- $\mathcal{R}_i$ is a sparse zero-sum reward function. Agent i receives $r^i = 1$ upon collecting a food item, and $r^i = 0$ otherwise.

2.3 Deep Q-learning

Q-learning is a value-based reinforcement learning algorithm that aims to learn the optimal action-value function $Q^*(s, a)$, which represents the maximum expected return achievable by following a policy π after taking action a in state s.

In Deep Q-learning, we approximate this function using a neural network with parameters θ. The objective is to minimize the Temporal Difference (TD) error between the predicted Q-values and the Bellman optimality target.

2.3.1 Optimization

The network is trained to minimize a loss function derived from the Bellman equation. For a transition tuple (s, a, r, s') sampled from a replay buffer $\mathcal{D}$, the loss is defined as:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[(y - Q(s, a; \theta))^2 \right] \tag{5}$$

where the target y is calculated as

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-) \tag{6}$$

and:

- $Q(s, a; \theta)$ is the predicted value of the online network.
- y is the TD target (the "ground truth" for this step).
- θ^- denotes the parameters of the target network, which are frozen and updated only periodically to stabilize training.
- γ is the discount factor.

This dependence on a target that depends on network parameters (θ^-) creates a "moving target" problem, which can lead to instability. The periodic update of the target network is a standard technique to mitigate this "chasing a non-stationary target" issue.

2.4 Double Deep Q-learning

Standard Q-learning is known to suffer from overestimation bias because the maximization step tends to prefer overestimated values. To mitigate this, we employ Double Deep Q-learning (DDQN).

This approach decouples action selection from action evaluation. The online network θ is used to select the greedy action, while the target network θ^- is used to estimate its value:

$$\mathcal{L}_{DDQN}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[\left(r + \gamma Q(s', \operatorname*{argmax}_{a'} Q(s', a'; \theta); \theta^-) - Q(s, a; \theta) \right)^2 \right] \tag{7}$$

where:

- $\operatorname*{argmax}_{a'} Q(s', a'; \theta)$ is the action selected by the online network.
- $Q(\cdot; \theta^-)$ evaluates the value of that selected action using the target network.

2.4.1 Huber Loss

To make the training more robust to large gradient updates from noisy rewards, we replace the standard Mean Squared Error (MSE) with the Huber Loss for the Bellman update.

The Huber loss is quadratic for small errors and linear for large errors, preventing exploding gradients. It is defined as:

$$L_\delta(e) = \begin{cases} \frac{1}{2}e^2 & \text{for } |e| \leq \delta, \\ \delta(|e| - \frac{1}{2}\delta) & \text{otherwise.} \end{cases} \tag{8}$$

where:

- $e = y - Q(s, a; \theta)$ is the TD error.
- δ is the threshold parameter.

2.5 Transformer

The Transformer is a deep learning architecture introduced by Vaswani et al. [16] that relies entirely on an attention mechanism to draw global dependencies between input and output. Unlike Recurrent Neural Networks (RNNs), Transformers process sequences in parallel rather than sequentially, allowing for significantly faster training and the capture of long-range dependencies.

2.5.1 Embeddings

In deep learning models, particularly those processing discrete data (such as words in NLP or grid coordinates in Reinforcement Learning), inputs are often categorical. Processing these directly as sparse, high-dimensional one-hot vectors is computationally inefficient and fails to capture semantic relationships between categories.

To address this, we employ embeddings [4]. An embedding is a mapping that projects discrete variables into a lower-dimensional continuous vector space. Formally, we define an embedding function $\text{Embed}(\cdot)$ as a learnable linear transformation applied to a one-hot encoded input.

For a discrete input represented as a one-hot vector $\boldsymbol{x} \in \mathbb{R}^N$ (where N is the number of distinct categories), the embedding is obtained by:

$$\text{Embed}(\boldsymbol{x}) = \boldsymbol{x}\boldsymbol{W}_E \tag{9}$$

where:

- $\boldsymbol{x}$ is the one-hot encoded input vector.
- $\boldsymbol{W}_E \in \mathbb{R}^{N \times d}$ is the learnable embedding matrix.
- d is the dimensionality of the embedding space.

During training, the weights in $\boldsymbol{W}_E$ are updated via backpropagation, allowing the model to learn representations where semantically similar inputs are located closer together in the vector space.

2.5.2 Self-Attention Mechanism

The core component of the Transformer is the Scaled Dot-Product Attention. The input consists of queries and keys of dimension d_k, and values of dimension d_v. To compute attention, the model packs the queries, keys, and values into matrices $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$, respectively.

The matrix of outputs is computed as follows:

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}}\right)\boldsymbol{V} \tag{10}$$

where:

- $\boldsymbol{Q}$ is the Query matrix.
- $\boldsymbol{K}$ is the Key matrix.
- $\boldsymbol{V}$ is the Value matrix.
- d_k is the scaling factor (dimension of the keys) used to prevent vanishing gradients in the softmax function.

2.5.3 Multi-Head Attention

To allow the model to mutually attend to information from different representation subspaces at different positions, the Transformer employs Multi-Head Attention. This mechanism projects Queries, Keys, and Values h times with different learned linear projections:

$$\text{MultiHead}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\boldsymbol{W}^O \tag{11}$$

where each head is defined as:

- $\text{head}_i = \text{Attention}(\boldsymbol{Q}\boldsymbol{W}_i^Q, \boldsymbol{K}\boldsymbol{W}_i^K, \boldsymbol{V}\boldsymbol{W}_i^V)$.
- $\boldsymbol{W}_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \boldsymbol{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \boldsymbol{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ are the parameter matrices for the projections of the i-th head.
- $\boldsymbol{W}^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ is the output projection matrix.
- h is the number of heads. In general, we choose $d_k = d_v = \frac{d_{\text{model}}}{h}$ to keep the computational cost similar to a single-head attention with full dimensionality.

This mechanism acts as a set of learnable projections. The input vectors are projected into h distinct representation subspaces using learned linear transition matrices.

2.5.4 Masked Self-Attention

When training the decoder, it is critical to preserve the auto-regressive property, ensuring that the prediction for a token at position t depends only on the known outputs at positions less than t.

To achieve this, we modify the self-attention mechanism by implementing masking. We introduce a mask matrix $\boldsymbol{M}$ to the scaled dot-product attention. This mask sets the attention scores corresponding to illegal connections (future tokens) to $-\infty$ before the softmax step:

$$\text{MaskedAttention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}} + \boldsymbol{M}\right)\boldsymbol{V} \tag{12}$$

where $\boldsymbol{M}$ is a square matrix defined as:

$$M_{ij} = \begin{cases} 0 & \text{if } i \geq j \text{ (allow attention)} \\ -\infty & \text{if } i < j \text{ (mask future)} \end{cases}$$

Because $\exp(-\infty) \approx 0$, the softmax function assigns zero probability to these future positions, effectively "hiding" them from the model during the attention computation.

2.5.5 Positional Encoding

Since the Transformer contains no recurrence and no convolution, the model itself is invariant to the order of the sequence. To inject information about the relative or absolute position of the tokens in the sequence, positional encodings are added to the input embeddings at the bottom of the encoder and decoder stacks.

The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. A common choice is to use sine and cosine functions of different frequencies:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \tag{13}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \tag{14}$$

where:

- pos is the position in the sequence.
- i is the dimension index.

Each dimension of positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$.

2.5.6 Position-wise Feed-Forward Networks

Each layer of the encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between:

$$\text{FFN}(\boldsymbol{x}) = \max(0, \boldsymbol{x}\boldsymbol{W}_1 + \boldsymbol{b}_1)\boldsymbol{W}_2 + \boldsymbol{b}_2 \tag{15}$$

Although the linear transformations are the same across different positions, they use different parameters from layer to layer.

2.5.7 Residual Connection and Layer Normalization

A key feature of the Transformer architecture is the use of residual connections followed by layer normalization around each of the sub-layers (attention and feed-forward networks). For any sub-layer function $\text{Sublayer}(\boldsymbol{x})$, the output is:

$$\text{Output} = \text{LayerNorm}(\boldsymbol{x} + \text{Sublayer}(\boldsymbol{x})) \tag{16}$$

This mechanism facilitates the flow of gradients through deep networks and stabilizes training.

2.5.8 Training Objective

The goal of the Transformer model during training is to minimize the difference between the predicted probability distribution and the actual target distribution. For sequence modeling tasks, this is formulated as the Cross-Entropy Loss:

$$\mathcal{L}_{\text{CE}} = -\sum_{t=1}^{T}\sum_{c=1}^{C} y_{t,c}\log(\hat{y}_{t,c}) \tag{17}$$

where:

- T is the sequence length.
- C is the number of classes.

- $y_{t,c}$ is the binary indicator if class c is the correct target at position t.
- $\hat{y}_{t,c}$ is the predicted probability that the target at position t belongs to class c.

A significant advantage of the Transformer architecture is its computational efficiency during this phase. Unlike RNNs, which must generate predictions sequentially, the Transformer utilizes the masking mechanism described above to process the entire sequence in parallel. This allows the gradient and loss to be computed for all time steps in a single forward pass.

2.5.9 Encoder Data Flow

To give a better understanding, we describe the flow of data tensors through a single Encoder Layer. Let the input sequence be represented by a matrix $\boldsymbol{X} \in \mathbb{R}^{T \times d_{\text{model}}}$, where T is the sequence length.

1. **Input Processing:** Raw tokens are embedded and summed with the positional encoding matrix $\boldsymbol{P}$, where $\boldsymbol{P}_{pos,i} = PE_{(pos,i)}$ as previously defined:

$$\boldsymbol{X}_0 = \text{Embed}(\text{Input}) + \boldsymbol{P}$$

2. **Adding context:** The sequence flows through the Multi-Head Attention mechanism, followed by a residual connection and layer normalization:

$$\boldsymbol{X}' = \text{LayerNorm}\Big(\boldsymbol{X}_0 + \text{MultiHead}(\boldsymbol{X}_0, \boldsymbol{X}_0, \boldsymbol{X}_0)\Big)$$

   In the encoder, $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ all originate from the output of the previous layer $\boldsymbol{X}_0$.

3. **Feature Extraction:** context-rich vectors are processed by the Feed-Forward Network:

$$\boldsymbol{Z} = \text{LayerNorm}\Big(\boldsymbol{X}' + \text{FFN}(\boldsymbol{X}')\Big)$$

The resulting matrix $\boldsymbol{Z}$ serves as the input to the next encoder layer. For the final layer of the stack, $\boldsymbol{Z}$ represents the fully context-rich history and serves as input to the decoder.

2.5.10 Decoder Data Flow

The decoder operates similarly to the encoder but includes an additional attention mechanism to integrate information from the encoder. Let $\boldsymbol{Y}$ be the decoder input (target sequence) and $\boldsymbol{Z}$ be the output of the final encoder layer described above.

1. **Masked Self-Attention:** The decoder first processes its own input $\boldsymbol{Y}$. To preserve the auto-regressive property, a mask is applied to the self-attention mechanism:

$$\boldsymbol{Y}' = \text{LayerNorm}\Big(\boldsymbol{Y} + \text{MaskedMultiHead}(\boldsymbol{Y}, \boldsymbol{Y}, \boldsymbol{Y})\Big)$$

2. **Encoder-Decoder Attention:** The decoder then attends to the encoder's output. Here, Queries ($\boldsymbol{Q}$) come from the decoder's previous step ($\boldsymbol{Y}'$), while Keys ($\boldsymbol{K}$) and Values ($\boldsymbol{V}$) come from the encoder output ($\boldsymbol{Z}$):

$$\boldsymbol{Y}'' = \text{LayerNorm}\Big(\boldsymbol{Y}' + \text{MultiHead}(\boldsymbol{Y}', \boldsymbol{Z}, \boldsymbol{Z})\Big)$$

3. **Processing:** Finally, the output is passed through the feed-forward network:

$$\text{Output} = \text{LayerNorm}\Big(\boldsymbol{Y}'' + \text{FFN}(\boldsymbol{Y}'')\Big)$$

2.6 Variational Autoencoder

Variational encoders (VAEs) are a class of probabilistic models that find latent low-dimensional representations of data. VAEs are thus a method for performing dimensionality reduction to reduce data down to their intrinsic dimensionality.

As their name suggests, VAEs are a type of autoencoder. An autoencoder is a model that takes a vector, $\boldsymbol{x}$, compresses it into a lower-dimensional vector, $\boldsymbol{z}$, and then decompresses $\boldsymbol{z}$ back into $\boldsymbol{x}$.

2.6.1 The Probabilistic Encoder

First, we define the encoding process:

$$\boldsymbol{\mu} = h_{\phi}^{(1)}(\boldsymbol{x}) \tag{18}$$

$$\log \boldsymbol{\sigma}^2 = h_{\phi}^{(2)}(\boldsymbol{x}) \tag{19}$$

where:

- $\boldsymbol{\mu}$ is the mean vector of the latent distribution.
- $\log \boldsymbol{\sigma}^2$ is the logarithm of the variance vector.
- $h_{\phi}^{(1)}, h_{\phi}^{(2)}$ are two neural networks parameterized by ϕ.
- $\boldsymbol{x}$ is the input vector.

Instead of mapping an input to a fixed vector, the encoder maps an input $\boldsymbol{x}$ to a distribution over the latent space. Using the parameters obtained above, the approximate posterior distribution $q_{\phi}(\boldsymbol{z} \mid \boldsymbol{x})$ is defined as a multivariate Gaussian distribution with a diagonal covariance matrix:

$$q_{\phi}(\boldsymbol{z} \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{z}; \boldsymbol{\mu}, \operatorname{diag}(\boldsymbol{\sigma}^2)) \tag{20}$$

where:

- $q_{\phi}(\boldsymbol{z} \mid \boldsymbol{x})$ is the probabilistic encoder (approximate posterior).
- $\operatorname{diag}(\boldsymbol{\sigma}^2)$ is the diagonal covariance matrix derived from the variance vector.

2.6.2 The Probabilistic Decoder

Next, we define the decoding process. We assume that the data are generated from a latent variable $\boldsymbol{z}$ via a distribution $p_{\theta}(\boldsymbol{x} \mid \boldsymbol{z})$. This distribution is parameterized by a neural network f_{θ} (the decoder).

For real-valued data, $p_{\theta}(\boldsymbol{x} \mid \boldsymbol{z})$ is typically modeled as a multivariate Gaussian:

$$p_{\theta}(\boldsymbol{x} \mid \boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}; f_{\theta}(\boldsymbol{z}), \sigma^2 \boldsymbol{I}) \tag{21}$$

where:

- $p_{\theta}(\boldsymbol{x} \mid \boldsymbol{z})$ is the probabilistic decoder (likelihood).
- $f_{\theta}(\boldsymbol{z})$ is a neural network parameterized by θ that outputs the mean of the data distribution.
- $\sigma^2 \boldsymbol{I}$ is the fixed covariance matrix, where $\boldsymbol{I}$ is the identity matrix.

To generate a new data point (or reconstruction) $\boldsymbol{x}'$, we first sample a latent vector $\boldsymbol{z}$ and then sample from the likelihood:

$$\boldsymbol{z} \sim q_{\phi}(\boldsymbol{z} \mid \boldsymbol{x}) \tag{22}$$

$$\boldsymbol{x}' \sim p_{\theta}(\boldsymbol{x} \mid \boldsymbol{z}) \tag{23}$$

2.6.3 Reparameterization

To allow for backpropagation through stochastic sampling of $\boldsymbol{z}$, we employ the reparameterization trick. Instead of sampling $\boldsymbol{z}$ directly, we sample an auxiliary noise variable $\boldsymbol{\epsilon}$:

$$\boldsymbol{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon} \tag{24}$$

where:

- $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ is an auxiliary noise variable sampled from a standard normal distribution.
- $\odot$ denotes the element-wise product.

2.6.4 Kullback-Leibler (KL) divergence

In mathematical statistics, the Kullback–Leibler (KL) divergence, also known as relative entropy, is a non-symmetric measure of the difference between two probability distributions P and Q. The KL divergence quantifies the information lost when Q is used to approximate the true distribution P.

For discrete probability distributions P and Q defined on the same probability space $\mathcal{X}$, the KL divergence is defined as:

$$D_{KL}(P \mid\mid Q) = \sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)} \tag{25}$$

where:

- $\mathcal{X}$ is the discrete sample space (the set of possible outcomes).
- $P(x)$ is the probability of outcome x in the true distribution.
- $Q(x)$ is the probability of outcome x in the approximating distribution.

2.6.5 Optimization

The network is trained by maximizing the Evidence Lower Bound (ELBO), which corresponds to minimizing the following loss function:

$$\mathcal{L}(\phi, \theta) = -\sum_{i=1}^{n} \left(\mathbb{E}_{\boldsymbol{z} \sim q_\phi(\boldsymbol{z} \mid \boldsymbol{x}_i)} \big[\log p_\theta(\boldsymbol{x}_i \mid \boldsymbol{z}) \big] - D_{KL}\big(p(\boldsymbol{z}) \mid\mid q_\phi(\boldsymbol{z} \mid \boldsymbol{x}_i)\big) \right) \tag{26}$$

where:

- $\mathcal{L}(\phi, \theta)$ is the loss function to be minimized.
- The first term is the Reconstruction Loss (negative log-likelihood).
- The second term is the Kullback-Leibler (KL) divergence Eq. (25), acting as a regularizer.
- $p(\boldsymbol{z}) = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ is the prior distribution over the latent space.

2.7 Conditional Variational Autoencoder

The Conditional Variational Autoencoder (CVAE) is an extension of the VAE that allows the model to generate data conditioned on specific auxiliary information. While a VAE models the latent variables solely based on the input data $\boldsymbol{x}$, a CVAE conditions both the encoder and the decoder on a condition vector $\boldsymbol{c}$.

2.7.1 The Conditional Encoder

The encoding process is modified to accept both the input $\boldsymbol{x}$ and condition $\boldsymbol{c}$. The neural networks now learn a mapping from the concatenated input $[\boldsymbol{x}, \boldsymbol{c}]$ to the latent space parameters:

$$\boldsymbol{\mu} = h_\phi^{(1)}(\boldsymbol{x}, \boldsymbol{c}) \tag{27}$$

$$\log \boldsymbol{\sigma}^2 = h_\phi^{(2)}(\boldsymbol{x}, \boldsymbol{c}) \tag{28}$$

where:

- $\boldsymbol{c}$ is the condition vector (e.g., historical trajectory).
- $[\boldsymbol{x}, \boldsymbol{c}]$ denotes the concatenation of the input and condition vectors.
- $h_\phi^{(1)}, h_\phi^{(2)}$ are neural networks conditioned on $\boldsymbol{c}$.

The approximate posterior distribution is now conditioned on $\boldsymbol{c}$:

$$q_\phi(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{c}) = \mathcal{N}(\boldsymbol{z}; \boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2)) \tag{29}$$

2.7.2 The Conditional Decoder

The decoding process also incorporates the condition vector. The decoder network f_θ takes the sampled latent variable $\boldsymbol{z}$ concatenated with $\boldsymbol{c}$ to reconstruct the input:

$$p_\theta(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{c}) = \mathcal{N}(\boldsymbol{x}; f_\theta(\boldsymbol{z}, \boldsymbol{c}), \sigma^2 \boldsymbol{I}) \tag{30}$$

where:

- $p_\theta(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{c})$ is the conditional likelihood.
- $f_\theta(\boldsymbol{z}, \boldsymbol{c})$ is the decoder network that outputs the mean of the data distribution given the latent vector and condition.

2.7.3 Conditional Objective Function

The reparameterization trick remains the same as in the standard VAE. However, the objective function is updated to reflect the conditioning. The network maximizes conditioned ELBO:

$$\mathcal{L}_{\mathrm{CVAE}}(\phi, \theta) = -\sum_{i=1}^{n} \left(\mathbb{E}_{\boldsymbol{z} \sim q_\phi(\boldsymbol{z} \mid \boldsymbol{x}_i, \boldsymbol{c}_i)} \big[\log p_\theta(\boldsymbol{x}_i \mid \boldsymbol{z}, \boldsymbol{c}_i) \big] - D_{KL}\big(p(\boldsymbol{z} \mid \boldsymbol{c}_i) \,||\, q_\phi(\boldsymbol{z} \mid \boldsymbol{x}_i, \boldsymbol{c}_i) \big) \right) \tag{31}$$

where:

- $\boldsymbol{c}_i$ is the condition vector corresponding to the i-th data point.
- $p(\boldsymbol{z} \mid \boldsymbol{c}_i)$ is the conditional prior distribution. Often, this is simplified to a standard normal $p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ if the latent space is assumed to be independent of the condition in the prior, or it can be learned by another network.

3 Opponent Model Architecture

Opponent modeling (OM) plays a pivotal role in improving robustness and directly addressing the non-stationarity inherent in Multi-Agent Reinforcement Learning (MARL) environments. By inferring the hidden states, intentions, or policies of other agents, a learning agent can adapt its strategy to exploit opponent mistakes or coordinate more effectively.

Beyond theoretical RL, opponent modeling has been successfully applied in various real-world domains, including professional sports analytics to predict player movement in football and basketball [20], military simulations for patrolling and formation movement [15], and AI for adaptive video games [3, 11].

3.1 Categorization of Opponent Modeling

Existing approaches to opponent modeling can be broadly categorized based on their prediction targets and methodology. Nashed and Zilberstein provide a comprehensive taxonomy [10], who divide the field into three primary streams:

1. **Action Prediction:** These methods attempt to directly approximate the opponent's policy function $\pi(a|s)$. The goal is to predict the next immediate action of the opponent. Classical approaches use frequency counting, while modern Deep RL approaches, such as Deep Reinforcement Opponent Network (DRON) [7], use neural networks to regress the opponent's actions. Machine Theory of Mind [13] extends this by learning a generalizable population model.

2. **Type or Strategy Classification:** Instead of predicting per-step actions, these methods classify the opponent into a set of predefined strategies or "types" (e.g., Aggressive vs. Defensive). Bayesian methods are often used here to maintain a belief distribution over possible types [1, 6].

3. **Intent and Goal Inference:** A more abstract approach focuses on predicting the long-term intent or subgoal of the opponent. This addresses the "short-sightedness" of action prediction. By knowing where the opponent is going, the agent can adapt even if the opponent's low-level policy changes.

Our work builds on the third category. Specifically, we aim to replicate and enhance the *Opponent Modeling based on Subgoal Inference* (OMG) framework [19]. While previous works have utilized Variational Autoencoders (VAEs) to model agent behavior [12, 21], our approach introduces a Transformer-based CVAE architecture to better capture long-range temporal dependencies.

Intuition: The concept of inferring subgoals mirrors human intuition, for example, in social settings. When we observe an agent's movement, we instinctively infer their destination based on context. For example, a student entering a library and walking resolutely toward the study area is inferred to have the subgoal "find a seat," whereas a student walking toward the cafeteria is inferred to have the subgoal "get food." Crucially, even if the second student takes a winding path to avoid a crowd, the underlying subgoal remains consistent. By modeling this high-level intent rather than the low-level steps, an agent can robustly predict opponent behavior even in novel situations.

3.2 Proposed Architecture

We propose a generative opponent model that infers the opponent's intent from their observed historical trajectory. Mathematically, we treat the "subgoal" not as a discrete symbol, but as a continuous latent vector $\boldsymbol{z}$ within a learned manifold.

The architecture is composed of two primary networks used during inference and two auxiliary modules used only during training.

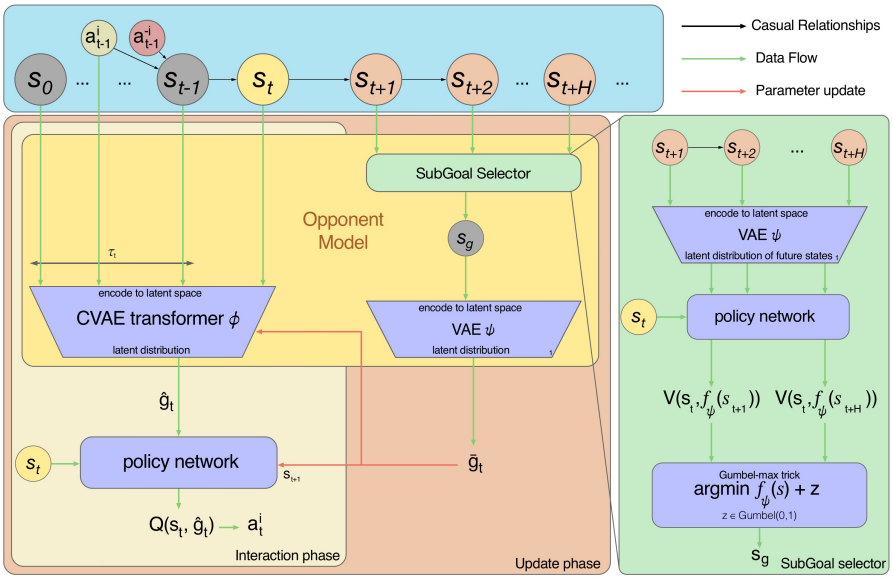

Figure 1: Overview of our Transformer-based Opponent Model. TODO: describe the image accurately

1. **Inference Model (The Student):** A Transformer-based Conditional Encoder that observes the history τ_t and the current state s_t to predict the posterior distribution of the subgoal latent vector $q_\phi(\hat{g}_t|\tau_t, s_t)$.

2. **Generative Model (The Decoder):** A Transformer Decoder that attempts to reconstruct the current state s_t given the predicted subgoal vector $\hat{g}_t$ and history.

3. **Pre-trained Oracle:** A standard VAE trained on future trajectories to learn the "true" latent representation of future subgoals. [TODO: use definition of ESS like: VAE learns a manifold $\mathcal{Z}$ that represents the space $\mathcal{P}$.]

4. **Subgoal Selector (The Teacher):** A mechanism that selects the most relevant future subgoal from the Oracle's distribution to supervise the inference model.

3.2.1 Inference Model

The core of our module is the inference model, parameterized by ϕ. It aims to approximate the posterior distribution of the opponent's subgoal $\hat{g}_t$, given their history.

Let $\tau_t = (s_{t-K}, a_{t-K}, \ldots, s_{t-1}, a_{t-1})$ be the trajectory history of length K. The encoder processes this sequence using a Transformer Encoder to produce a latent distribution:

$$q_\phi(\hat{g}_t \mid s_t, \tau_t) = \mathcal{N}(\boldsymbol{\mu}_\phi(s_t, \tau_t), \boldsymbol{\sigma}^2_\phi(s_t, \tau_t)) \tag{32}$$

where $\boldsymbol{\mu}_\phi$ and $\boldsymbol{\sigma}_\phi$ are the outputs of the Transformer's projection heads. This distribution represents the agent's uncertainty about the opponent's current intent.

3.2.2 Generative Model

To ensure that the latent variable $\hat{g}_t$ captures meaningful information about the state, we employ a decoder parameterized by θ. The decoder attempts to reconstruct the current state s_t conditioned on the sampled subgoal $\hat{g}_t$ and the history τ_t.

$$p_\theta(s_t \mid \hat{g}_t, \tau_t) \tag{33}$$

In our implementation, this is a Transformer Decoder that attends to the history τ_t and the subgoal query $\hat{g}_t$ to minimize the reconstruction error of the state tensor.

3.2.3 Pre-training the Oracle

To supervise the Inference Model, we require a "ground truth" definition of what the opponent's subgoal actually is. Since subgoals are latent, we cannot label them manually. First, we train a standard VAE on trajectories (using either random or trained policy) to learn an accurate latent space representation of the environment. The Oracle is then used in Subgoal Selector to take a future state s_{t+h}, where $h \in H$ is the subgoal horizon, and encodes it into a distribution $p_\psi(\bar{g}_t|s_{t+h})$. Oracle is trained purely to reconstruct future states, ensuring that the latent space $\mathcal{Z}$ captures meaningful spatial and semantic features of the state of the game.

3.2.4 Subgoal Selector

During the training of the Opponent Model, we cannot simply use a fixed future state as the target, because the opponent might have multiple valid subgoals or the horizon may vary. The **Subgoal Selector** is a mechanism that dynamically selects the best "target" latent vector from the candidate future states.

Given a window H of future states $\{s_{t+1}, \ldots, s_{t+H}\}$, the Oracle first encodes each state into a candidate latent vector $f_\psi(s_{t+h})$. The Subgoal Selector then evaluates these candidates using our agent's value function $V(s)$. In a zero-sum setting, we assume the opponent intends to reach the state that minimizes our value.

To simulate this selection stochastically, we employ the **Gumbel-Min trick**. We define the index h^* of the selected future state as:

$$h^* = \underset{h \in \{1,\ldots,H\}}{\operatorname{argmin}} \Big(V(s_{t+h}) - \tau_{temp} \cdot G_h \Big) \tag{34}$$

where:

- $V(s_{t+h})$ is the value of the future state h steps ahead, estimated by the agent's critic.
- $G_h \sim \text{Gumbel}(0, 1)$ is noise sampled from the standard Gumbel distribution, calculated as $-\log(-\log(u))$ where $u \sim \text{Uniform}(0, 1)$.
- $\tau_{temp} > 0$ is the temperature parameter that controls the randomness of the selection.

The ground truth subgoal $\bar{g}_t$ is then defined as the encoding of this selected state:

$$\bar{g}_t = f_\psi(s_{t+h^*}) \tag{35}$$

This results in a target distribution $p_\psi(\bar{g}_t \mid s_{t+h^*})$ that serves as the supervision signal for the inference model, effectively teaching the student to predict the most dangerous future state the opponent actually reached.

3.3 Training Objective

The Opponent Model is trained to maximize a modified ELBO Eq. (31). Our objective is twofold:

1. **Reconstruction:** The latent subgoal $\hat{g}$ must contain enough information to reconstruct the current state s_t.
2. **Alignment:** The predicted subgoal distribution q_ϕ must match the target subgoal distribution p_ψ selected by the Subgoal Selector.

The loss function is defined as:

$$\mathcal{L}_{OM}(\phi, \theta) = \mathbb{E}_{q_\phi(\hat{g}_t|\tau_t, s_t)}\Big[\log p_\theta(s_t \mid \hat{g}_t, \tau_t)\Big] - \beta D_{KL}\Big(p_\psi(\bar{g}_t \mid s_{t+h}) \mid\mid q_\phi(\hat{g}_t \mid \tau_t, s_t)\Big) \tag{36}$$

where:

- $\hat{g}_t$ is the latent subgoal vector sampled from the inference model at a time t.
- $\bar{g}_t$ is the target latent vector identified by the Subgoal Selector at a time t.
- The first term is reconstruction loss between the input state and the reconstructed state.
- The second term minimizes the divergence between the predicted intent and the actual future outcome.

During execution time, the Oracle and Subgoal Selector are discarded. The agent relies solely on the inference model q_ϕ to predict the latent subgoal vector $\hat{g}$ from the observed history.

4 Implementation Details

To evaluate the proposed architecture, we implement a Double Deep Q-Network (DDQN) agent that utilizes the inferred subgoals from the Opponent Model to condition its value estimation. This section details the state representation, the Convolutional Neural Network (CNN) architecture used for feature extraction, and the training algorithm.

4.1 State Representation and CNNs

In grid-world environments like the Foraging Game, the state s_t preserves critical spatial information. Specifically, the relative positions of agents, resources, and obstacles. Standard Multi-Layer Perceptrons (MLPs) require flattening the input into a 1D vector, which destroys this spatial locality and makes learning translation-invariant features inefficient.

To address this, we employ Convolutional Neural Networks (CNNs) [9]. The input state is represented as a tensor of shape (B, H, W, F), where:

- B is the batch size.
- $H = 11, W = 11$ are the grid height and width.
- $F = 4$ is the number of feature channels (Empty, Food, Agent 1, Agent 2).

The feature extractor $f_{\text{CNN}}(s_t)$ consists of 3 convolutional blocks. Each block applies a 2D convolution followed by a Rectified Linear Unit (ReLU) activation function. The final convolutional map is flattened to produce a dense spatial embedding vector $\mathbf{h}_{state}$.

4.2 Integrating Subgoals into Q-Learning

The standard Q-function $Q(s, a)$ estimates the value of taking action a in state s. In our approach, we augment this function to condition it on the opponent's inferred intent. The modified Q-function is denoted as $Q(s, a, \hat{g})$, where $\hat{g}$ is the latent subgoal vector predicted by the Opponent Model.

The integration is performed via feature concatenation. The spatial embedding $\mathbf{h}_{state}$ is concatenated with the inferred subgoal vector $\hat{g}_t$, sampled from the inference model q_ϕ, and explicit features of the opponent's current location. This combined vector is fed into a Dueling Q-Network [18] head to predict action values:

$$\mathbf{h}_{joint} = \text{Concat}(f_{\text{CNN}}(s_t), \hat{g}_t) \tag{37}$$

$$Q(s, a, \hat{g}_t) \approx f_{duel}(\mathbf{h}_{joint}) \tag{38}$$

4.3 Algorithm Pseudocode

The training process involves two concurrent optimization loops: the Opponent Model (OM) learning to reconstruct states and infer subgoals, and the DDQN agent learning to maximize rewards given those subgoals.

Algorithm 1 outlines the joint training procedure. Note that the Opponent Model is trained using the Subgoal Selector to ensure semantic consistency, while the Agent utilizes the Inference Model during interaction to maintain causality.

Algorithm 1 Q-Learning with Subgoal Inference (OM-DDQN)

Initialize:
  Replay Buffer $\mathcal{D}$
  Agent Network Q_θ, Target Network Q_{θ^-}
  Opponent Model: Inference q_ϕ, Decoder p_θ, Oracle f_ψ
Pre-train: Train Oracle f_ψ on random trajectories to learn latent space $\mathcal{Z}$.
for episode $= 1$ to M **do**
  Initialize state s_0 and history $\tau_0 = \emptyset$
  for step $t = 0$ to T **do**
    Inference:
      Update history: $\tau_t \leftarrow \text{Append}(\tau_{t-1}, s_t)$
      Sample subgoal: $\hat{g}_t \sim q_\phi(\cdot \mid s_t, \tau_t)$
    Action:
      Select a_t using ϵ-greedy policy on $Q_\theta(s_t, \cdot, \hat{g}_t)$
    Step:
      Execute a_t, observe reward r_t and next state s_{t+1}
      Store tuple $(s_t, a_t, r_t, s_{t+1}, \tau_t)$ in $\mathcal{D}$
  Training Phase:
  Sample batch of transitions and sequences from $\mathcal{D}$
  // 1. Train Opponent Model (The Objective)
  Sample future states s_{t+h} for the batch from $\mathcal{D}$
  Select target $\bar{g}_t$ using Subgoal Selector via Eq. (35)
  Compute $\mathcal{L}_{OM}$ via Eq. (36)
  Update ϕ, θ via gradient descent
  // 2. Train Q-Agent (The Policy)
  Calculate target $y = r + \gamma \max_{a'} Q_{\theta^-}(s', a', \hat{g}_{t+1})$
  Compute Loss $\mathcal{L}_{DDQN} = \text{Huber}(y - Q_\theta(s, a, \hat{g}_t))$
  Update θ via gradient descent
  Update target networks $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$

4.4 Hyperparameters

The hyperparameters used for training the Opponent Model and the Q-Learning agent are detailed in Table 1. These values were selected based on preliminary tuning and the architectural constraints of the Foraging environment.

5 Experiments

We designed our experiments to evaluate the hypothesis that conditioning a Q-learning agent on inferred subgoals accelerates convergence and improves stability compared to a standard model-free baseline. The evaluation is conducted in the **Foraging Environment** described in Section 2.2.2.

Table 1: Hyperparameters for Opponent Model (OM) and Q-Learning Agent (DQN)

Component	Parameter	Value
Environment	Grid Size ($H \times W$)	11×11
	Max Steps per Episode	50
	State Feature Channels	4
	Action Space Size	4
	Subgoal Horizon H	3
Transformer OM	Model Dimension (d_{model})	256
	Feedforward Dimension	1024
	Attention Heads	4
	Encoder Layers	1
	Decoder Layers	1
	Latent Dimension (z)	32
	Dropout	0.12
	VAE Learning Rate	1×10^{-4}
	CVAE Learning Rate	1×10^{-4}
Loss Weights	VAE β (KL Weight)	0.1
	CVAE $\beta_{start} \rightarrow \beta_{end}$	$0.0 \rightarrow 2.0$
	Selector Temperature $\tau_{start} \rightarrow \tau_{end}$	$2.0 \rightarrow 0.1$
Q-Learning Agent	Q-Network Hidden Dimension	128
	Learning Rate	3×10^{-4}
	Discount Factor γ	0.955
	Batch Size	16
	Replay Buffer Capacity	1,000
	Target Update Frequency	1,000 steps
	Exploration $\epsilon_{start} \rightarrow \epsilon_{end}$	$1.0 \rightarrow 0.05$
	Soft Update τ_{soft}	0.005

5.1 Experimental Setup

The environment consists of a grid with two agents spawned at fixed symmetric locations (first and last column, middle row) and two stationary food items (first and last row, middle column). The distance from each agent to either food item is identical.

- **Objective:** Agents must visit a food location to collect it.
- **Rewards:** +1 for collection, 0 otherwise.
- **Dynamics:** Agents cannot occupy the same cell. Agent 1 (the learning agent) executes actions first, conferring a slight strategic advantage in race conditions.

The opponent follows one of two fixed deterministic policies, selected randomly at the start of each episode:

1. **Top-Priority:** Navigate to the top food via the Y-axis, then the X-axis.
2. **Bottom-Priority:** Navigate to the bottom food via the Y-axis, then the X-axis.

The objective of the Opponent Model (OM) is to predict the opponent's coordinates up to H steps into the future, identifying which target is being pursued.

5.2 Evaluation of Transformer-based Subgoal Inference

Our first set of experiments evaluated the full end-to-end architecture where the Transformer-based OM infers subgoals from raw history.

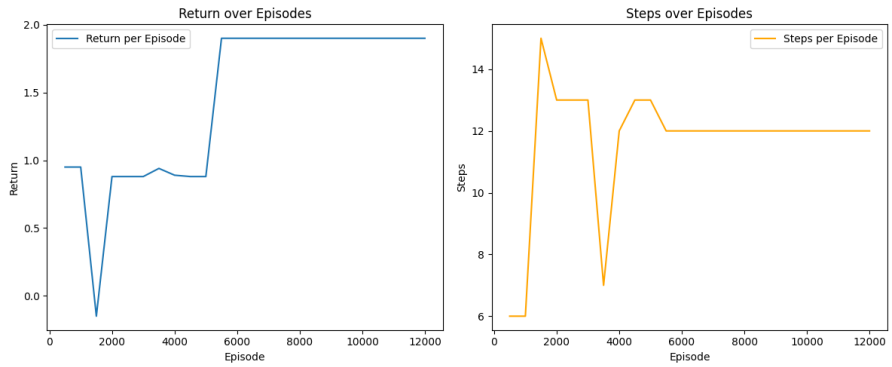

Figure 2: Training progress of Q-learning agent with pre-programmed Oracle. Agent want's to pick up both food points resulting in reward of 2

We observed significant optimization difficulties with the Transformer architecture in this domain:

1. **Computational Cost:** Training the Transformer-based inference model required 4–8 hours, compared to minutes for standard MLPs.

2. **Posterior Collapse:** The inference model struggled to balance the reconstruction loss (current state) with the KL-divergence term (future subgoal). The model frequently converged to a trivial identity mapping, predicting the previous state s_{t-1} rather than the future state s_{t+h}. This indicates that the immediate history was a stronger signal for minimizing reconstruction error than the latent future subgoal.

3. **Instability:** The joint optimization of the OM and the Q-network proved unstable. While a baseline Q-learning agent (using simple linear layers) solved the environment in approximately $20,000$ episodes, the OM-augmented agent failed to converge to a winning policy.

5.3 Hypothesis Testing with a Synthetic Oracle

Given the optimization challenges of the Transformer, we pivoted to an isolation study. We aimed to validate whether the concept of subgoal conditioning is beneficial, assuming an ideal Inference Model.

We replaced the learned OM with a pre-programmed Oracle. This Oracle has privileged knowledge of the opponent's strategy. Once the opponent makes a definitive move (up or down), the Oracle generates a "perfect" latent subgoal vector $\hat{g} \in \mathbb{R}^{32}$.

- If Opponent targets Top: $\hat{g} = \mathbf{v} \cdot 1$
- If Opponent targets Bottom: $\hat{g} = \mathbf{v} \cdot (-1)$
- If Unknown: $\hat{g} = \mathbf{0}$

where $\mathbf{v}$ is a fixed random vector with values drawn from $[-1, 1]$.

5.3.1 Feature Fusion and Dimensionality Mismatch

In this phase, we upgraded the Q-network to use CNNs for state processing. The baseline CNN-DDQN (without subgoals) converged to an optimal policy in approximately $6,000$ episodes.

However, incorporating the Oracle's signal unexpectedly degraded performance. The Oracle-augmented agent required approximately $8,000$ to $12,000$ episodes to converge and exhibited less robust behavior.

We identified a critical issue in Multimodal Feature Fusion:

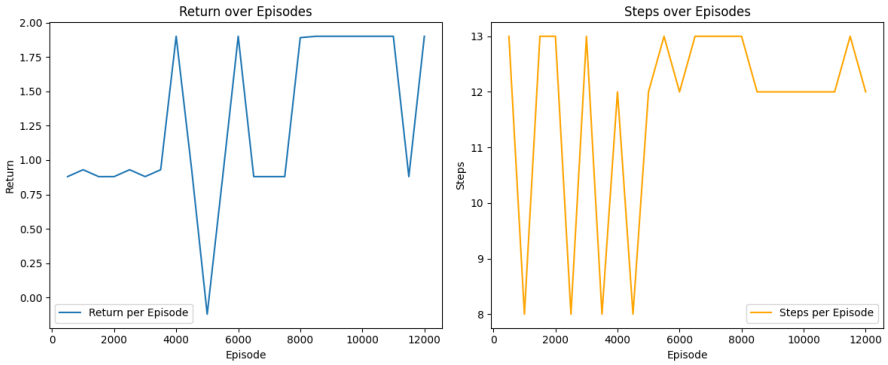

Figure 3: Training progress of Q-learning agent with pre-programmed Oracle. Agent want's to pick up both food points resulting in reward of 2

- **Dimensionality Imbalance:** The CNN feature extractor output a dense vector of size $3,872$, while the subgoal vector was size 32. When concatenated, the subgoal signal was effectively drowned out by the high-dimensional state noise, leading the network to treat the explicit intent as negligible noise.
- **Distribution Mismatch:** The CNN outputs pass through a ReLU activation (range $[0, \infty)$), while the subgoal vector uses Tanh (range $[-1, 1]$).

To mitigate this, we introduced a Projection Head: a linear layer followed by a Tanh activation that projects the $3,872$-dimensional CNN output down to 32 dimensions before concatenation. This ensured that both modality vectors had equal size and value range.

Despite these architectural corrections, the subgoal-conditioned agent still trained slower than the baseline. This suggests that for low-complexity tasks like Foraging, the overhead of processing auxiliary goal features adds learnable parameters that complicate the optimization landscape more than the semantic value of the goal aids it.

6 Conclusion

In this thesis, we proposed a novel Opponent Modeling architecture based on ESS games. We formalized the ESS domain, decomposing state spaces into context and subgoal manifolds, and designed a Transformer-based CVAE to infer latent intentions from historical trajectories.

While the theoretical framework offers a rigorous way to distinguish concrete subgoals from abstract states, our empirical results in the Foraging domain highlight significant implementation challenges. We found that Transformer-based inference models are prone to posterior collapse in simple Markovian environments, often defaulting to predicting immediate history rather than long-term intent. Furthermore, our evaluation using a pre-programmed Oracle revealed that naively conditioning Q-networks on explicit subgoals does not guarantee performance gains. In fact, without careful feature fusion, high-dimensional state representations can overwhelm the semantic signal provided by the opponent model.

6.1 Future Work

Our experiments demonstrated that conditioning Q-learning on auxiliary subgoal features introduces significant computational overhead, naturally resulting in slower convergence compared to simpler baselines. Consequently, we propose shifting the core hypothesis of future research from faster learning to exploitation and policy robustness.

Despite the negative empirical results, this work contributes a formal definition of ESS domains and identifies critical pitfalls in fusing latent intent vectors with model-free Reinforcement Learning.

References

[1] Stefano V. Albrecht and Peter Stone. Autonomous agents modelling other agents: A comprehensive survey and open problems. *Artif. Intell.*, 258:66–95, 2018.

[2] Tim Baarslag, Mark J. C. Hendrikx, Koen V. Hindriks, and Catholijn M. Jonker. Learning about the opponent in automated bilateral negotiation: a comprehensive survey of opponent modeling techniques. *Auton. Agents Multi Agent Syst.*, 30(5):849–898, 2016.

[3] Sander C. J. Bakkes, Pieter H. M. Spronck, and H. Jaap van den Herik. Opponent modelling for case-based adaptive game AI. *Entertain. Comput.*, 1(1):27–37, 2009.

[4] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Janvin. A neural probabilistic language model. *J. Mach. Learn. Res.*, 3:1137–1155, 2003.

[5] Filippos Christianos, Lukas Schäfer, and Stefano V. Albrecht. Shared experience actor-critic for multi-agent reinforcement learning. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.

[6] David Churchill and Michael Buro. Portfolio greedy search and simulation for large-scale combat in starcraft. In *2013 IEEE Conference on Computational Inteligence in Games (CIG), Niagara Falls, ON, Canada, August 11-13, 2013*, pages 1–8. IEEE, 2013.

[7] He He and Jordan L. Boyd-Graber. Opponent modeling in deep reinforcement learning. In Maria-Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of the 33nd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 1804–1813. JMLR.org, 2016.

[8] Michael L. Littman. Markov games as a framework for multi-agent reinforcement learning. In *Proceedings of the Eleventh International Conference on Machine Learning*, pages 157–163. Morgan Kaufmann, 1994.

[9] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin A. Riedmiller, Andreas Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nat.*, 518(7540):529–533, 2015.

[10] Samer B. Nashed and Shlomo Zilberstein. A survey of opponent modeling in adversarial domains. *J. Artif. Intell. Res.*, 73:277–327, 2022.

[11] Santiago Ontañón, Kinshuk Mishra, Neha Sugandh, and Ashwin Ram. Case-based planning and execution for real-time strategy games. In Rosina Weber and Michael M. Richter, editors, *Case-Based Reasoning Research and Development, 7th International Conference on Case-Based Reasoning, ICCBR 2007, Belfast, Northern Ireland, UK, August 13-16, 2007, Proceedings*, volume 4626 of *Lecture Notes in Computer Science*, pages 164–178. Springer, 2007.

[12] Georgios Papoudakis, Filippos Christianos, and Stefano V. Albrecht. Agent modelling under partial observability for deep reinforcement learning. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 19210–19222, 2021.

[13] Neil C. Rabinowitz, Frank Perbet, H. Francis Song, Chiyuan Zhang, S. M. Ali Eslami, and Matthew M. Botvinick. Machine theory of mind. In Jennifer G. Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, volume 80 of *Proceedings of Machine Learning Research*, pages 4215–4224. PMLR, 2018.

[14] Lloyd S. Shapley. Stochastic games. *Proceedings of the National Academy of Sciences*, 39(10):1095–1100, 1953.

[15] Emiel Ubink, Frank Aldershoff, Wouter Lotens, and Arend Woering. Behavior modeling through chaos for simulation of dismounted soldier operations. In *Modeling and Simulation for Military Operations III*, volume 6965, pages 30–41. SPIE, 2008.

[16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 5998–6008, 2017.

[17] John von Neumann and Oskar Morgenstern. *Theory of Games and Economic Behavior (60th-Anniversary Edition)*. Princeton University Press, 2007.

[18] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In Maria-Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of the 33nd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, volume 48 of *JMLR Workshop and Conference Proceedings*, pages 1995–2003. JMLR.org, 2016.

[19] Xiaopeng Yu, Jiechuan Jiang, and Zongqing Lu. Opponent modeling based on subgoal inference. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[20] Yisong Yue, Patrick Lucey, Peter Carr, Alina Bialkowski, and Iain A. Matthews. Learning fine-grained spatial models for dynamic sports play prediction. In Ravi Kumar, Hannu Toivonen, Jian Pei, Joshua Zhexue Huang, and Xindong Wu, editors, *2014 IEEE International Conference on Data Mining, ICDM 2014, Shenzhen, China, December 14-17, 2014*, pages 670–679. IEEE Computer Society, 2014.

[21] Luisa M. Zintgraf, Kyriacos Shiarlis, Maximilian Igl, Sebastian Schulze, Yarin Gal, Katja Hofmann, and Shimon Whiteson. Varibad: A very good method for bayes-adaptive deep RL via meta-learning. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.